

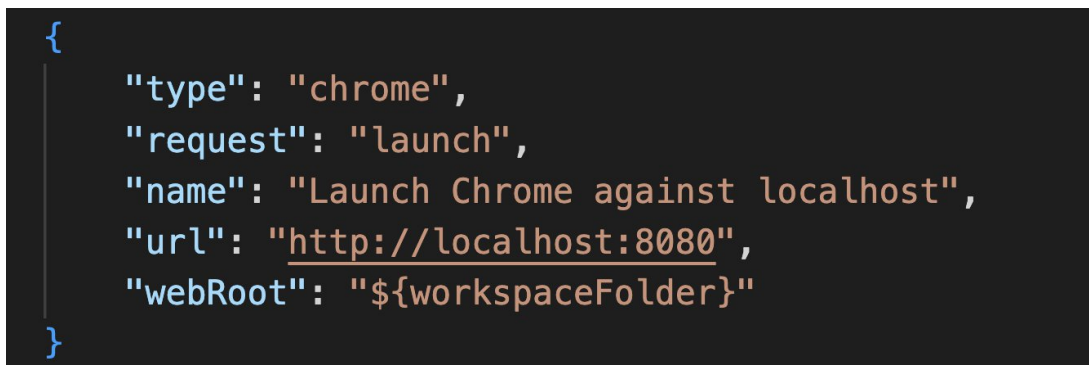
Практичне заняття 2

Робота із форматом даних json, робота із запитамися - метод `fetch()`. React.

JSON (JavaScript Object Notation) – стандартний текстовий формат для представлення структурованих даних на основі синтаксису об'єкта JavaScript. Він зазвичай використовується для передачі даних у веб-додатках (наприклад, відправлення деяких даних із сервера клієнту, таким чином, щоб це могло відображатися на веб-сторінці або навпаки).

JSON - текстовий формат даних, який слідує за синтаксисом об'єкта JavaScript, який був популяризований Дугласом Крокфордом. Незважаючи на те, що він дуже схожий на синтаксис об'єкта JavaScript, його можна використовувати незалежно від JavaScript, і багато програмних середовищ мають можливість читати (аналізувати) і генерувати JSON.

JSON -файл може мати наступний вигляд(рисунок 2.1)



```
{
  "type": "chrome",
  "request": "launch",
  "name": "Launch Chrome against localhost",
  "url": "http://localhost:8080",
  "webRoot": "${workspaceFolder}"
}
```

Рисунок 2.1 - Стандартний вигляд JSON -файлу

Для роботи із JSON JavaScript надає наступні методи:

- `JSON.stringify` для перетворення об'єктів у JSON.
- `JSON.parse` для перетворення JSON назад на об'єкт.

Приклад перетворення об'єкта у json-формат

```
let student = {
  name: 'John',
  age: 30,
  course: 'JS',
  today_theme: ['json', 'fetch', 'promise']
};
```

```
let json = JSON.stringify(student)
console.log(typeof json);
```

console.log(json);

Результат виконання - рисунок 2.2.

```
[Running] node "/Users/kateryna_z/Desktop/Orda/test/html/tempCodeRunnerFile.js"
string
{"name":"John","age":30,"course":"JS","today_theme":["json","fetch","promise"]}
```

Рисунок 2.2. - результат виконання коду

Щоб декодувати JSON-рядок, нам потрібен інший метод під назвою `JSON.parse`.

let value = JSON.parse(str, [reviver]);

```
let
student='{ "name": "John", "age": 30, "course": "JS", "today_theme": ["json", "fetc
h", "promise"] }'
let obj=JSON.parse(student)
console.log(obj)
```

Рисунок 2.3. - результат виконання коду

```
[Running] node "/Users/kateryna_z/Desktop/Orda/test/html/tempCodeRunnerFile.js"
{
  name: 'John',
  age: 30,
  course: 'JS',
  today_theme: [ 'json', 'fetch', 'promise' ]
}
```

JavaScript може надсилати мережні запити на сервер і підвантажувати нову інформацію за необхідності.

Наприклад, ми можемо використовувати мережний запит, щоб:

- Відправити замовлення,
- Завантажити інформацію про користувача,
- Запитати останні оновлення з сервера..

Метод **`fetch()`** – сучасний і дуже потужний. Він не підтримується старими браузерами (можна використовувати поліфіл).

let promise = fetch(url, [options])

- url – URL для надсилання запиту.
- options – додаткові параметри: метод, заголовки тощо.

Без options це простий GET-запит, що завантажує вміст на адресу url.

Браузер відразу починає запит і повертає проміс, який зовнішній код використовує для отримання результату.

Процес отримання відповіді зазвичай відбувається у два етапи.

По-перше, promise виконується з об'єктом вбудованого класу Response як результат, як тільки сервер надішле заголовки відповіді.

На цьому етапі ми можемо перевірити статус HTTP-запиту і визначити, чи він успішний, а також подивитися заголовки, але поки без тіла відповіді.

Проміс завершується з помилкою, якщо fetch не зміг виконати HTTP-запит, наприклад, при помилці мережі або якщо немає такого сайту. HTTP-статуси 404 та 500 не є помилкою.

так як, метод fetch повертає проміс, то нього так як і до промісів можна застосовувати методи **.then**, **.catch**, **.finally**.

В якості URL для надсилання запиту, використовуються можливості API.

API – це механізми, які дозволяють двом програмним компонентам взаємодіяти один з одним, використовуючи набір визначень та протоколів. Наприклад, система метеослужби містить щоденні дані про погоду. Програма погоди на телефоні спілкується з цією системою через API і показує щоденні оновлення погоди на телефоні.

API - Application Programming Interface, що означає програмний інтерфейс програми. У контексті API слово «додаток» стосується будь-якого ПЗ з певною функцією. Інтерфейс можна розглядати як сервісний контракт між двома програмами. Цей контракт визначає, як вони взаємодіють один з одним, використовуючи запити та відповіді. Документація API містить інформацію про те, як розробники повинні структурувати ці запити та відповіді.

Для тренування ми будемо використовувати відкриті для використання API.

Наприклад, запит для отримання інформації за допомогою fetch може виглядати наступним чином:

```
1 fetch('https://jsonplaceholder.typicode.com/todos/5')
2   .then(response => response.json())
3   .then(json => console.log(json))
4   .catch(() => console.log('ERROR'))
5   .finally(() => console.log('I will always fulfill'))
```

Стрічка 1- ми звертаємось до ресурсу <https://jsonplaceholder.typicode.com/>, щоб отримати певні дані про лист завдань, а саме про завдання під номером 5.

Стрічка 2 - за умови успішного виконання запиту ми отримуємо відповідь у форматі json (ми застосовуємо .json)

Стрічка 3 - відображаємо отримані дані

Стрічка 4 - блок catch, допомагає зреагувати на reject(проміс виконався із помилкою)

Стрічка 5 - завжди буде виконана без залежності від того, як саме виконався проміс.

```
{
  userId: 1,
  id: 5,
  title: 'laboriosam mollitia et enim quasi adipisci quia provident illum',
  completed: false
}
I will always fulfill
```

Рисунок 2.4. - результат виконання коду

Завдання на практичну роботу №2

Рівень 1

1. Відображення даних про себе:

- Створіть файл HTML з елементом <script>.
- Використовуйте fetch для завантаження JSON-файлу з вашими даними (ім'я, вік, хобі).
- Розпарсіть JSON-файл за допомогою JSON.parse().
- Використовуйте document.createElement() для створення елементів HTML, що відображають ваші дані.

2. Список покупок:

- Створіть файл HTML з елементом <script>.
- Використовуйте fetch для завантаження JSON-файлу зі списком покупок.
- Розберіть JSON-файл за допомогою JSON.parse().
- Створіть список HTML з елементами для кожного пункту списку покупок.

3. Прогноз погоди (текстовий):

- Створіть файл HTML з елементом <script>.

- Використовуйте `fetch` для отримання даних про погоду з API (наприклад, OpenWeatherMap).
- Розберіть JSON-файл з відповіддю API.
- Відобразіть поточну температуру та опис погоди на веб-сторінці.

4. Цитати дня (випадкова):

- Створіть файл HTML з елементом `<script>`.
- Використовуйте `fetch` для завантаження JSON-файлу з цитатами дня.
- Розберіть JSON-файл за допомогою `JSON.parse()`.
- Виберіть випадкову цитату з масиву цитат.
- Відобразіть випадкову цитату на веб-сторінці.

5. Відображення зображення:

- Створіть файл HTML з елементом `<script>`.
- Використовуйте `fetch` для завантаження JSON-файлу з URL зображення.
- Створіть елемент `<img>` з URL зображення.
- Додайте елемент `<img>` на веб-сторінку.

6. Зміна кольору тексту:

- Створіть файл HTML з елементом `<script>` та кнопкою.
- Використовуйте `fetch` для завантаження JSON-файлу з кольором (наприклад, `#FF0000`).
- Розберіть JSON-файл за допомогою `JSON.parse()`.
- Змініть колір тексту кнопки на основі отриманого кольору.

7. Відображення даних про тварин:

- Створіть файл HTML з елементом `<script>`.
- Використовуйте `fetch` для завантаження JSON-файлу з даними про тварин (назва, тип, фото).
- Розберіть JSON-файл за допомогою `JSON.parse()`.
- Створіть список HTML з елементами `<li>` для кожної тварини, що містить назву та фото.

8. Прогноз погоди (з іконкою):

- Створіть файл HTML з елементом `<script>`.
- Використовуйте `fetch` для отримання даних про погоду з API (наприклад, OpenWeatherMap).
- Розберіть JSON-файл з відповіддю API.
- Відобразіть поточну температуру, опис погоди та іконку погоди на веб-сторінці.

9. Цитати дня (за категорією):

- Створіть файл HTML з елементом `<script>` та випадаючим списком.
- Використовуйте `fetch` для завантаження JSON-файлу з цитатами дня та категоріями.
- Розберіть JSON-файл за допомогою `JSON.parse()`.
- Заповніть випадаючий список категоріями.
- При виборі категорії відобразіть цитату дня з цієї категорії.

10. Відображення рандомного факту:

- Створіть файл HTML з елементом `<script>`.
- Використовуйте `fetch` для завантаження JSON-файлу з фактами.
- Розберіть JSON-файл за допомогою `JSON.parse()`.
- Виберіть випадковий факт з масиву фактів.
- Відобразіть випадковий факт на веб-сторінці.

Рівень 2

1. Парсер JSON

Напишіть програму, яка приймає JSON-рядок в якості вхідних даних і розбиває його на окремі елементи (об'єкти, масиви, значення).

- Використовуйте вбудовані функції мови програмування для роботи з JSON.

- Виведіть результат у зручному форматі (наприклад, деревоподібна структура, таблиця).

2. Фільтр JSON

Напишіть програму, яка приймає JSON-файл та значення фільтра в якості вхідних даних.

- Програма повинна фільтрувати дані JSON, залишаючи лише ті елементи, які відповідають заданому значенню фільтра.
- Виведіть відфільтровані дані JSON.

3. Генератор JSON

Напишіть програму, яка генерує JSON-файл з заданими даними.

- Використовуйте вбудовані функції мови програмування для створення JSON-структури.
- Збережіть JSON-файл на диск.

4. Візуалізатор JSON

Напишіть програму, яка приймає JSON-файл в якості вхідних даних і візуалізує його у зручному форматі.

- Використовуйте бібліотеки візуалізації даних або вбудовані функції мови програмування.
- Відобразіть JSON-дані у вигляді таблиці, графіка або іншого формату.

5. Перевірка JSON

Напишіть програму, яка перевіряє, чи даний текст є дійсним JSON.

- Використовуйте вбудовані функції мови програмування або бібліотеки для перевірки JSON.
- Виведіть результат перевірки (дійсний JSON / недійсний JSON).

Рівень 3

1. Створіть веб-додаток для відображення списку користувачів. Кожен користувач повинен включати ім'я та електронну пошту. Додайте можливість позначати користувачів у відповідності до їх груп - ЦА, постійні клієнти та інші.

2. Розробіть веб-додаток з функціоналом фільтрації списку завдань за їхнім статусом (завершено чи незавершено).

3. Створіть веб-додаток для відстеження фільмів. Додайте можливість додавання нових фільмів та їхню рецензію.

4. Розробіть веб-додаток для обміну повідомленнями між користувачами. Додайте можливість створення чат-кімнат та відправлення повідомлень.

5. Розробіть веб-додаток інтерактивну дошку завдань, на якій користувачі можуть додавати завдання та присвоювати їм теги або мітки.

6. Створіть інтерактивну карту, де користувачі можуть додавати маркери для подій (наприклад, концертів, фестивалів, виставок тощо).

7. Створіть веб-додаток, що дозволяє користувачам залишати відгуки та оцінки для різних продуктів або послуг. Додайте можливість позначати не якісні товари або послуги.

8. Створіть веб-додаток для керування бібліотекою книг, де користувачі можуть додавати та видаляти книги. Додайте можливість користувачам робити помітки - вибране, прочитане, бажане і т.д.

**Рівень 4 (оберіть одне із завдань на вибір, головна умова - щоб не більше 3 людей з групи вибрали одне й те саме завдання)
Доповніть завдання з рівня 3 одним із наступних:**

1. Створіть новий компонент в який інтегруйте роботу з публічним API (наприклад, JSONPlaceholder) та виведіть реальні дані на вашому веб-сайті. Інформація має бути дотичною до вашого завдання з рівня 3.

2. Додайте можливість фільтрації, сортування і пошуку.

3. Додайте різні типи візуалізації (графіки, діаграми, таблиці) ваших даних

4. Зробіть ваш веб-сайт доступним для людей з інвалідністю, використовуючи принципи ARIA та WCAG.

5. Доповніть ваш додаток можливістю реєстрації та управління профілем користувача (зміна пароля, інформації).

6. Для кожного елемента у вашому списку додайте можливість переходу на окрему сторінку, де будуть відображені деталі цього елемента.

7. Доповніть ваш веб-додаток пагінацією, щоб розділити дані на сторінки. Надайте можливість вибору кількості елементів, що відображаються на 1 сторінці - користувачу.

8. Розширте свій додаток, додавши форму, де користувачі можуть ввести дані для нового елемента(в новому компоненті)

9. Додайте можливість взаємодії з популярними сторонніми сервісами через API, наприклад, сервісами геолокації, соціальних мереж або платіжних систем.

10. Розширте ваш додаток, додавши підтримку різних мов та локалей.

За результатами виконання практичного заняття сформувати звіт із кодом всіх створених файлів та додати скріншоти виконання завдань.

Контрольні запитання.

1. Що таке JSON?
2. Які основні типи даних в JSON?
3. Як правильно формувати JSON файл?
4. Як парсувати JSON файл в JavaScript?
5. Які бібліотеки можна використовувати для роботи з JSON в JavaScript?
6. Як валідувати JSON файл?
7. Як додати коментарі до JSON файлу?
8. Як отримати значення ключа, якщо воно не існує в JSON файлі?
9. Як перетворити JSON файл в інший формат даних (наприклад, CSV)?
10. Як використовувати JSON Schema для опису структури JSON даних?
11. Що таке метод fetch()?
12. Які аргументи можна використовувати в методі fetch()?
13. Як обробити відповідь fetch()?
14. Як перевірити, чи успішно виконано запит fetch()?
15. Які типи помилок можуть виникнути при використанні fetch()?
16. Як відправити POST запит з JSON даними за допомогою fetch()?
17. Як додати автентифікацію до fetch запиту?
18. Як обробити великі JSON файли за допомогою fetch()?
19. Як скасувати fetch запит?
20. Як використовувати fetch() для завантаження файлів?
21. Як використовувати fetch() для отримання даних з API?
22. Що таке React?
23. Яка структура компонентів React?
24. Як використовувати JSX в React?
25. Як рендерити дані JSON в React?
26. Як оновити компонент React, коли дані JSON змінюються?